

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

МЭДЭЭЛЭЛ ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

МЭДЭЭЛЛИЙН ТЕХНОЛОГИЙН ТЭНХИМ



Ө.Хүслэн

**Нийгмийн сүлжээний өгөгдлийн
автомат цуглуулга ба таамаглалт
шинжилгээний систем**

Sentiment Analysis ба ойролцоогоор хэрэглэгч оролцооны

таамаглал

БАКАЛАВРЫН ТӨГСӨЛТИЙН АЖИЛ

Улаанбаатар хот

2026 он

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

МЭДЭЭЛЭЛ ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

МЭДЭЭЛЛИЙН ТЕХНОЛОГИЙН ТЭНХИМ

**Нийгмийн сүлжээний өгөгдлийн автомат
цуглуулга ба таамаглалт шинжилгээний систем**

Sentiment Analysis ба ойролцоогоор хэрэглэгч оролцооны

таамаглал

Ажлын төрөл:	Бакалаврын төгсөлтийн ажил
Оюутан:	Ө.Хүслэн
Мэргэжлийн индекс:	D061303
Мэргэжил:	Мэдээллийн систем
Удирдагч:	Мөнхбуян
Зөвлөгч:	
Гүйцэтгэгч:	Ө.Хүслэн, B221930045

Улаанбаатар хот

2026 он

Батлав. Мэдээллийн технологийн тэнхим-ийн эрхлэгч:

..... /...../

Удирдагч:

..... /Мөнхбуян/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Сэдэв: “Нийгмийн сүлжээний өгөгдлийн автомат
цуглуулга ба таамаглалт шинжилгээний систем”

№	Ажлын бүлэг, хэсгийн нэр	Эзлэх хувь	Дуусах хугацаа
1	Удиртгал, асуудлын тодорхойлолт	5%	2026-02
2	Онолын хэсэг, хэрэглэгдэх техно- логи	20%	2026-03
3	Судалгааны арга зүй, системийн архитектур	25%	2026-03
4	Системийн хэрэгжилт ба туршилт	35%	2026-04
5	Үр дүн, дүгнэлт, тайлангийн бо- ловсруулалт	15%	2026-05

Төлөвлөгөө боловсруулсан оюутан: /Ө.Хүслэн/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Гарын үсэг
1	Удиртгал, төслийн зорилго ба хүрээ	2026-02	
2	Онолын судалгаа, ижил төстэй системийн харьцуулалт	2026-03	
3	Backend, scraper, database архитектурын боловсруулалт	2026-03	
4	React dashboard, AI report, export workflow хэрэгжилт	2026-04	
5	Туршилт, үр дүн, тайлангийн эцсийн боловсруулалт	2026-05	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

Удирдагч: /Мөнхбуян/

ЗӨВШӨӨРӨЛ

Оюутан Ө.Хүслэн-ын бичсэн “Нийгмийн сүлжээний өгөгдлийн автомат
цуглуулга ба таамаглалт шинжилгээний систем” сэдэвт төгсөлтийн ажлыг
улсын шалгалтын комисст хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: /...../

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

МЭДЭЭЛЭЛ ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

ШҮҮМЖИЙН ХУУДАС

Мэдээллийн технологийн тэнхим-ийн төгсөх курсын оюутан Ө.Хүслэн-ын
“Нийгмийн сүлжээний өгөгдлийн автомат цуглуулга ба таамаглалт шинжил-
гээний систем” сэдэвт төгсөлтийн ажлын шүүмж.

1. Төслөөр дэвшүүлсэн асуудал, түүнтэй холбоотой онолын материал ун-
шиж судалсан байдал.

.....
.....
.....
.....

2. Төслийн ерөнхий агуулга, шийдсэн зүйлүүд, хүрсэн үр дүн, харьцуулалт
ба дүгнэлт.

.....
.....
.....
.....

3. Эмх цэгцтэй, стандарт хангасан байдлаар диплом бичих шаардлагыг
биелүүлсэн эсэх.

.....
.....
.....

.....

4. Төслөөр орхигдуулсан болон дутуу болсон зүйлүүд, цаашид анхаарах зүйлүүд.

.....

.....

.....

.....

5. Ерөнхий оноо. (5 оноо)

.....

Шүүмж бичсэн: /...../

Ажлын газар:

Хаяг, утас:

Зохиогч эрхийн хамгаалал

Миний бие Ө.Хүслэн, “Нийгмийн сүлжээний өгөгдлийн автомат цуглуулга ба таамаглалт шинжилгээний систем” сэдэвт энэхүү төгсөлтийн ажил нь миний өөрийн бүтээл бөгөөд дараахыг нотолж байна. Үүнд:

- Энэхүү ажлыг боловсролын зэрэг авах зорилгоор бүхэлд нь буюу голлон өөрөө гүйцэтгэсэн.
- Ажлын аль нэг хэсгийг өөр байгууллага, сургуульд өмнө нь зэрэг, мэргэшил авах зорилгоор илгээгээгүй.
- Бусдын хэвлүүлсэн ажил, өгөгдөл, санаа, эх кодоос ашигласан тохиолдолд эх үүсвэрийг тодорхой заасан.
- Судалгааны болон хэрэгжилтийн явцад тусалсан эх үүсвэр, зөвлөгөө, дэмжлэгийг зохих байдлаар дурдсан.
- Хамтарсан ажил ашигласан бол тухайн хувь нэмрийг ялган тодорхойлсон.

Гарын үсэг: _____

Огноо: _____

Хураангуй

Энэхүү дипломын ажил нь Facebook хуудасны олон нийтэд харагдах нийтлэл, сэтгэгдэл, зураг, оролцооны үзүүлэлтийг автоматаар цуглуулж, хадгалж, хяналтын самбар болон AI тусламжтай тайлангаар ашиглах системийг боловсруулахад чиглэсэн. Бодит хэрэгжилт нь React/Vite frontend, FastAPI backend, Selenium ба BeautifulSoup-д суурилсан Facebook scraper, SQLAlchemy ORM бүхий PostgreSQL хадгалалт, TextBlob болон scikit-learn-д суурилсан үндсэн шинжилгээ, Google Gemini API-д суурилсан тайлан үүсгэх хэсгүүдээс бүрдэнэ.

Судалгааны үндсэн зорилгууд:

- Facebook хуудасны зорилтот жагсаалтыг удирдаж, scraper процессыг backend-ээс эхлүүлэх боломжтой болгох
- Нийтлэл, сэтгэгдэл, зураг болон likes, shares, comment count зэрэг оролцооны өгөгдлийг хадгалах
- Цуглуулсан өгөгдөл дээр TextBlob fallback бүхий sentiment analysis, LDA topic modeling, engagement score тооцоолол хийх
- React dashboard-аар live backend өгөгдлийг харуулах, backend холбогдохгүй үед demo fallback ашиглах
- Gemini API ашиглан сүүлийн өгөгдлөөс уншигдахуйц тайлан, зөвлөмж үүсгэх

Систем нь Facebook төвтэй өгөгдөл цуглуулах pipeline, REST API, өгөгдлийн сан, frontend dashboard, scraper target management, automation log viewer, AI report generation гэсэн бодит хэрэгжсэн бүрэлдэхүүнтэй. Энэхүү

хувилбар нь repository-д бодитоор байгаа Facebook төвтэй workflow-г л үндсэн боломж гэж авч үзсэн.

Түлхүүр үгс: Facebook scraper, React dashboard, FastAPI, PostgreSQL, SQLAlchemy, TextBlob, LDA, Gemini, social media analytics

Талархал

Энэхүү төгсөлтийн ажлыг гүйцэтгэх явцад судалгааны чиглэл, системийн архитектур, хэрэгжилтийн хүрээг тодорхойлоход зөвлөгөө өгч дэмжсэн удирдагч багш Мөнхбуян-д гүн талархал илэрхийлье. Мөн суралцах хугацаанд мэдээллийн технологийн онол, практикийн мэдлэг олгож, мэргэжлийн ур чадварыг хөгжүүлэхэд хувь нэмэр оруулсан Мэдээлэл холбооны технологийн сургууль-ийн багш нартаа талархал илэрхийлж байна.

Судалгаа, хөгжүүлэлтийн явцад дэмжлэг үзүүлсэн гэр бүл, найз нөхөд болон төслийн туршилт, бичиг баримтын боловсруулалтад тусалсан бүх хүнд баярлалаа.

Товчилсон үгс

API	Application Programming Interface
AI	Artificial Intelligence
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DOM	Document Object Model
JSON	JavaScript Object Notation
LDA	Latent Dirichlet Allocation
NLP	Natural Language Processing
ORM	Object-Relational Mapping
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UML	Unified Modeling Language
ERD	Entity-Relationship Diagram

Гарчиг

Зохиогч эрхийн хамгаалал	I
Хураангуй	II
Талархал	IV
1 Оршил	1
1.1 Судалгааны үндэслэл	1
1.2 Судалгааны зорилго	1
1.3 Судалгааны асуулт	2
1.4 Судалгааны таамаглал	2
1.5 Судалгааны ач холбогдол	2
1.6 Судалгааны хүрээ ба хязгаарлалт	3
1.7 Дипломын ажлын бүтэц	3
2 Онолын судалгаа	5
2.1 Нийгмийн сүлжээний өгөгдөл цуглуулах архитектур	5
2.2 Ижил төстэй системийн судалгаа	5
2.2.1 Commercial social listening tools	5
2.2.2 API-only collection	6
2.2.3 Browser automation scraper	6
2.3 Sentiment, Topic, Engagement Analysis онол	7
2.3.1 Sentiment Analysis (Сэтгэл хөдлөлийн шинжилгээ)	7
2.3.2 Topic Modeling (Сэдвийн загварчлал)	7
2.3.3 Engagement Score (Оролцооны үнэлгээ)	8
2.4 Хиймэл оюун (AI) ба Том хэлний загвар (LLM)	9
2.4.1 RAG (Retrieval-Augmented Generation) төстэй аргачлал	9
2.5 Бүлгийн дүгнэлт	10
3 Системийн шинжилгээ ба загварчлал	11
3.1 Хэрэглэгчийн шаардлага ба Use Case диаграм	11
3.1.1 Функциональ шаардлагууд	11
3.1.2 Функциональ бус шаардлагууд	12
3.1.3 Use Case тодорхойлолтууд	13
3.2 Системийн архитектурын загварчлал	15
3.3 Өгөгдлийн сангийн загварчлал (ERD)	16
3.4 Процессын дарааллын загварчлал	17
3.4.1 Өгөгдөл цуглуулах процесс (Activity)	17
3.4.2 Дарааллын диаграм (Sequence Diagram)	18

3.4.3	Тайлан үүсгэх процесс (AI Generation)	18
3.5	Бүлгийн дүгнэлт	19
4	Системийн хөгжүүлэлт	20
4.1	Төслийн бүтэц	20
4.2	Өгөгдлийн сангийн хөгжүүлэлт	20
4.3	Backend хөгжүүлэлт (FastAPI)	22
4.4	Scraper хөгжүүлэлт	22
4.5	Frontend хөгжүүлэлт (React Dashboard)	24
4.6	AI тайлан болон ML шинжилгээний хөгжүүлэлт	24
4.6.1	Data Analyzer модуль	24
4.6.2	Gemini AI тайлан	24
4.7	Бүлгийн дүгнэлт	26
5	Хэрэгжилт ба үр дүн	27
5.1	Хэрэгжилтийн ерөнхий үр дүн	27
5.2	Хэрэглэгчийн интерфэйс болон функцүүдийн хэрэгжилт	28
5.2.1	Data Sources болон Scraper Control	28
5.2.2	Үндсэн Dashboard болон Үзүүлэлтүүд	28
5.2.3	Admin Control болон Хяналт	29
5.3	Туршилтын үр дүн ба баталгаажуулалт	30
5.4	Үр дүнгийн хэлэлцүүлэг	31
5.5	Хязгаарлалт ба эрсдэл	31
6	Дүгнэлт	32
6.1	Үндсэн дүгнэлт	32
6.2	Шинэлэг тал	32
6.3	Цаашдын хөгжүүлэлтийн чиглэл	33
A	Хавсралт	34
A.1	Системийн тохиргооны жишээ	34
A.2	API Endpoint жишээ	34
A.3	Өгөгдлийн сангийн схем	35
B	Эх кодын жагсаалт	36
B.1	Үндсэн API сервер (api_server.py)	36
B.2	React Dashboard (App.tsx)	38
B.3	ML Analyzer (analyzer.py)	39

Зургийн жагсаалт

3.1	Хэрэглэгчийн үүрэг ба Use Case диаграм	12
3.2	Системийн логик архитектурын диаграм	16
3.3	Өгөгдлийн сангийн ERD диаграм	17
3.4	Scraper ажиллуулах процессын дарааллын диаграм	18
5.1	Data Sources цонх болон Automation Console	28
5.2	Үндсэн Dashboard болон Engagement trend	29
5.3	Admin Control болон Backend Status	29
5.4	Collection limits (Хязгаарлалтын тохиргоо)	30

Хүснэгтийн жагсаалт

2.1	Төслийн шийдлийн харьцуулалт	6
3.1	UC-01: Dashboard харах	13
3.2	UC-02: Scraper ажиллуулах	14
3.3	UC-03: AI тайлан үүсгэх	15
4.1	Backend endpoint-уудын үндсэн бүлэг	22
5.1	Хэрэгжсэн модулиудын нэгтгэл	27
5.2	API workflow баталгаажуулалт	30
A.1	Хавсралтын endpoint жагсаалт	34

БҮЛЭГ 1

Оршил

1.1 Судалгааны үндэслэл

Олон нийтийн Facebook хуудас дээр үүсэж буй нийтлэл, сэтгэгдэл, реакцийн хэмжээ нь хэрэглэгчдийн сонирхол, санал хандлага, контентийн үр нөлөөг ойлгоход хэрэгтэй өгөгдөл болдог. Гэвч уг өгөгдлийг гараар цуглуулах нь цаг их зарцуулдаг, давтагдах боломж муутай, олон хуудас дээр тогтмол хянахад хүндрэлтэй. Иймээс scraper, backend API, өгөгдлийн сан, dashboard, AI тайланг нэгтгэсэн систем шаардлагатай.

Энэ төслийн бодит хэрэгжилт нь Facebook өгөгдөл дээр төвлөрсөн. Frontend нь live backend өгөгдлийг харуулж, scraper target list буюу `pages.txt`-ийг удирдаж, scraper process эхлүүлэх, log харах, backend health шалгах, recent posts болон engagement trend харах боломжтой. Backend нь FastAPI endpoint-уудаар өгөгдөл, статистик, sentiment, topic, trend, export болон scraper control функцийг өгдөг.

1.2 Судалгааны зорилго

Энэхүү дипломын ажлын зорилго нь Facebook хуудасны нийтлэл, сэтгэгдэл, зураг, оролцооны үзүүлэлтийг автоматаар цуглуулах, хадгалах, шинжлэх, хэрэглэгчид dashboard болон AI тайлан хэлбэрээр хүргэх систем боловсруулах явдал юм.

Системийн бодит зорилтууд:

1. Facebook page URL/ID жагсаалтыг frontend болон backend-ээр удирдах
2. Selenium WebDriver ашиглан Facebook хуудаснаас пост, сэтгэгдэл, зураг, engagement metadata цуглуулах
3. PostgreSQL/SQLAlchemy ашиглан `facebook_posts`, `post_comments`, `post_images`, `analysis_results` хүснэгтүүдэд хадгалах
4. FastAPI ашиглан posts, comments, images, stats, sentiment, topics, trends, scraper control endpoint-ууд гаргах

5. React/Vite dashboard ашиглан live data, demo fallback, chart, query, backend health, scraper logs харуулах
6. Google Gemini API ашиглан сүүлийн өгөгдлөөс тайлан, зөвлөмж үүсгэх

1.3 Судалгааны асуулт

1. Facebook хуудаснаас нийтлэл, сэтгэгдлийг browser automation ашиглан давтагдах боломжтой байдлаар цуглуулж болох уу?
2. Цуглуулсан өгөгдлийг REST API болон dashboard-аар хэрэглэгчид ойлгомжтой байдлаар хүргэж болох уу?
3. TextBlob, LDA topic modeling, engagement score болон Gemini report generation-ийг нэг pipeline-д нэгтгэснээр судалгааны болон контент хяналтын шийдвэрт хэрэгтэй мэдээлэл гаргаж болох уу?

1.4 Судалгааны таамаглал

- **H1:** Facebook scraper, relational storage, FastAPI endpoint, React dashboard-ыг нэгтгэснээр өгөгдөл цуглуулах ба хянах ажлын гараар хийх ачааллыг бууруулна.
- **H2:** Likes, shares, comments, text length, hashtag count зэрэг feature-д суурилсан engagement score нь постуудыг харьцуулахад хэрэглэхүйц индикатор болно.
- **H3:** Gemini-д өгөгдлийн товч summary дамжуулах нь dashboard-ийн тоон үзүүлэлтийг уншигдахуйц зөвлөмж болгон хувиргахад тусална.

1.5 Судалгааны ач холбогдол

- Facebook хуудасны өгөгдөл цуглуулах, хадгалах, шалгах үйлдлийг нэг workflow болгоно.

- Маркетинг, судалгаа, контент хяналтын ажилд dashboard болон export боломж өгнө.
- Browser automation, REST API, ORM, frontend visualization, AI report generation-ийг нэг системд нэгтгэсэн жишээ болно.

1.6 Судалгааны хүрээ ба хязгаарлалт

Хүрээ:

- Facebook хуудасны нийтлэл, сэтгэгдэл, зураг, engagement metadata цуглуулах
- PostgreSQL үндсэн хадгалалт, SQLite тохиргооны fallback боломж
- FastAPI REST endpoint, Bearer token хамгаалалттай v1 endpoint-ууд
- React/Vite frontend dashboard, data source manager, admin query, Gemini report UI
- TextBlob/basic sentiment, LDA topic modeling, keyword trend extraction, engagement score

Хязгаарлалт:

- Бусад платформын scraper бодитоор хэрэгжээгүй.
- Deep-learning sentiment model болон graph-based interaction analysis үндсэн хэрэгжилтэд ашиглагдаагүй.
- Sentiment analysis нь голчлон англи текстэд илүү тохиромжтой; Монгол хэлний тусгай загвар бэлтгээгүй.
- Facebook UI болон access policy өөрчлөгдвөл scraper selector, login, modal handling дахин тохируулах шаардлагатай.

1.7 Дипломын ажлын бүтэц

Энэхүү дипломын ажил нь нийт 6 бүлгээс бүрдэнэ:

1-р бүлэг: Оршил - судалгааны үндэслэл, зорилго, асуулт, таамаглал, ач холбогдол, хүрээ болон хязгаарлалтыг тодорхойлно.

2-р бүлэг: Онолын судалгаа - өгөгдөл цуглуулах архитектур, API, ML шинжилгээ, ижил төстэй системийн харьцуулалт болон онолын үндэслэлүүд.

3-р бүлэг: Системийн шинжилгээ ба загварчлал - системийн архитектур, UML диаграмууд (use case, дараалал, класс), өгөгдлийн сангийн загвар.

4-р бүлэг: Системийн хөгжүүлэлт - frontend, backend, scraper болон AI хэсгүүдийн бодит кодчилал, хэрэгжилтийн шийдлүүд.

5-р бүлэг: Хэрэгжилт ба үр дүн - системийн ажиллагааны үр дүн, хэрэглэгчийн интерфэйс (UI), тайлан гаргалт.

6-р бүлэг: Дүгнэлт - судалгааны ерөнхий үр дүн, шинэлэг тал, цаашдын хөгжүүлэлтийн боломжууд.

БҮЛЭГ 2

Онолын судалгаа

2.1 Нийгмийн сүлжээний өгөгдөл цуглуулах архитектур

Нийгмийн сүлжээний өгөгдөл нь ихэвчлэн динамик ачаалагддаг тул энгийн HTTP хүсэлтээр бүх контент, comment modal, lazy-loaded post list бүрэн авах боломжгүй. Ийм нөхцөлд Selenium WebDriver зэрэг browser automation хэрэгсэл ашиглан бодит browser session үүсгэж, scroll, navigation, modal interaction хийх нь практик шийдэл болдог.

BeautifulSoup нь Selenium-оор ачаалсан HTML/DOM бүтцээс текст, link, image URL, metadata зэрэг мэдээллийг ялгах parser-ийн үүрэгтэй. Энэ хоёр технологийг хослуулснаар JavaScript-heavy page дээр browser control болон content parsing-ийг салгаж зохион байгуулах боломжтой.

2.2 Ижил төстэй системийн судалгаа

Зах зээлд нийгмийн сүлжээний дата аналитикийн олон төрлийн шийдлүүд байдаг ч тэдгээр нь ихэвчлэн enterprise түвшний өндөр үнэтэй, эсвэл API хязгаарлалттай байдаг.

2.2.1 Commercial social listening tools

Brandwatch, Sprout Social, Hootsuite зэрэг платформууд нь олон нийтийн сүлжээний мэдээллийг цуглуулж шинжилдэг enterprise шийдлүүд юм.

- **Давуу тал:** Олон платформ, enterprise dashboard-тай, sentiment болон topic analysis-г гүнзгийрүүлэн хийдэг.
- **Сул тал:** Үнэтэй, data access/API хязгаарлалттай. Хувь хүн эсвэл жижиг төсөлд ашиглахад өртөг өндөр.

2.2.2 API-only collection

Facebook Graph API зэрэг албан ёсны API ашиглан өгөгдөл цуглуулах арга.

- **Давуу тал:** Тогтвортой, дүрмийн дагуу access хийгддэг тул IP block орох эрсдэл бага.
- **Сул тал:** Facebook public page data-д API access маш хязгаарлагдмал. Заавал баталгаажсан аппликэйшн, хуудасны админ эрх шаардана.

2.2.3 Browser automation scraper

Selenium, Puppeteer зэрэг хэрэгслийг ашиглан вэб хөтчийг автоматжуулах арга.

- **Давуу тал:** Динамик UI-тай ажиллах боломжтой, API эрх шаардахгүйгээр public өгөгдлийг цуглуулах бүрэн боломжтой.
- **Сул тал:** Facebook-ийн UI өөрчлөгдөхөд selector засвар шаардана, хурд харьцангуй удаан.

Хүснэгт 2.1: Төслийн шийдлийн харьцуулалт

Шийдэл	Давуу тал	Хязгаарлалт
Commercial tools	Олон платформ, найдвартай	Үнэтэй, API хязгаарлалттай
API-only collection	Тогтвортой	Зөвшөөрөл авах хэцүү, өгөгдөл бага
Browser automation	Public өгөгдлийг шууд авна	UI өөрчлөлтөд эмзэг
Энэхүү төсөл	Scraper, API, DB, dashboard, AI report нэг workflow-д орсон, үнэгүй хөгжүүлэх боломжтой	Facebook төвтэй, production benchmark dataset байхгүй

2.3 Sentiment, Topic, Engagement Analysis онол

2.3.1 Sentiment Analysis (Сэтгэл хөдлөлийн шинжилгээ)

Текстээс эерэг, сөрөг, саармаг хандлагыг тодорхойлох үйл явц юм. TextBlob нь polarity (эерэг/сөрөг) болон subjectivity (объектив/субъектив) тооцох хөнгөн sentiment analysis хэрэгсэл юм. Хэрэв TextBlob байхгүй бол basic keyword matching fallback ашиглаж болно. Энэ нь өндөр нарийвчлалтай deep learning model биш боловч эхний шатны dashboard insight-д хангалттай baseline өгдөг.

Сэтгэл хөдлөлийн шинжилгээг хийхэд байгалийн хэл боловсруулах (Natural Language Processing - NLP) технологиуд чухал үүрэг гүйцэтгэдэг. Энэхүү технологи нь текстийг үгсийн сангийн түвшинд задалж (tokenization), үг бүрийн язгуур болон утгын жинг (weight) тооцдог. Текст дэх эерэг болон сөрөг утгатай үгсийн харьцаа, өгүүлбэрийн бүтэц зэргээс хамааран эцсийн оноог (polarity score) гаргаж авдаг. Polarity нь ихэвчлэн -1.0 (маш сөрөг) -ээс +1.0 (маш эерэг) хооронд хэлбэлздэг.

Монгол хэл зэрэг морфологийн баялаг бүтэцтэй хэлний хувьд sentiment analysis хийх нь илүү нарийн бөгөөд төвөгтэй байдаг. Үүнд залгавар, дагавар, мөн ярианы хэлний онцлог (жишээлбэл, сленг, эможи) ихээр нөлөөлдөг тул энгийн keyword matching нь хангалтгүй байж болно. Иймд TextBlob нь англи хэл дээр өндөр нарийвчлалтай байх боловч, цаашид BERT зэрэг Transformer загварт суурилсан дэвшилтэт аргачлалыг нэвтрүүлэх хэрэгцээ бий.

2.3.2 Topic Modeling (Сэдвийн загварчлал)

Topic modeling-д CountVectorizer болон Latent Dirichlet Allocation (LDA) ашигласнаар постын текстүүдээс давтагдах keyword cluster гаргаж болно. Энэ нь нийт өгөгдлөөс хамгийн их яригдаж буй сэдвүүдийг автоматаар илрүүлэх машин сургалтын unsupervised арга юм.

Latent Dirichlet Allocation (LDA) аргачлалын математик үндэслэл: LDA нь баримт бичгүүд (documents) дэх үгсийн тархалтыг магадлалын онолд тулгуурлан тооцоолдог загвар юм. Энэ нь документ бүрийг тодорхой сэдвүүдийн магадлалт холимог (mixture of topics) гэж үздэг бөгөөд сэдэв бүр нь тодорхой үгсийн магадлалт холимог (mixture of words) байдаг гэж таамагладаг.

Үүнийг математик талаас нь дараах байдлаар томъёолдог:

$$p(\theta, z, w|\alpha, \beta) = p(\theta|\alpha) \prod_{n=1}^N p(z_n|\theta)p(w_n|z_n, \beta)$$

Энд:

- w_n нь ажиглагдсан үг.
- z_n нь тухайн үг аль сэдэвт харьяалагдаж буйг илтгэх хувьсагч.
- θ нь документ бүрийн сэдвийн тархалт.
- α болон β нь Dirichlet тархалтын параметрууд.

LDA алгоритм нь эдгээр далд хувьсагчдын (latent variables) тархалтыг Gibbs Sampling эсвэл Variational Inference аргаар тооцож, хамгийн их магадлалтай сэдвүүдийг (topics) илрүүлдэг. Энэхүү төсөлд Scikit-learn сангийн хэрэгжилтийг ашигласан.

2.3.3 Engagement Score (Оролцооны үнэлгээ)

Engagement analysis-д likes, shares, comments зэрэг observable metric-ээс нийлмэл оноо гаргах нь постуудыг хооронд нь харьцуулах энгийн боловч ойлгомжтой арга юм. Оролцооны оноог дараах байдлаар томъёолж болно:

$$EngagementScore = w_1 \times Likes + w_2 \times Comments + w_3 \times Shares$$

Энд w_i нь тухайн үйлдлийн ач холбогдлын жин (weight) юм.

2.4 Хиймэл оюун (AI) ба Том хэлний загвар (LLM)

Google Gemini API зэрэг LLM (Large Language Model) үйлчилгээ нь structured summary data-г уншигдахуйц тайлан, зөвлөмж болгон хувиргах боломж өгдөг. Энэ төсөлд Gemini нь scraper эсвэл database-ийн оронд ажиллахгүй; харин dashboard дээрх recent posts-ийн товч нэгтгэлийг тайлбарлах AI давхарга (AI Layer) болж ашиглагдана.

Том хэлний загварууд (Large Language Models - LLM) нь тэрбум гаруй параметртэй, асар их хэмжээний текст өгөгдөл дээр сургагдсан мэдрэлийн сүлжээний загварууд юм. Тэдгээр нь хүнтэй төстэй байдлаар текст үүсгэх, орчуулах, нэгтгэн дүгнэх, асуултанд хариулах зэрэг олон төрлийн даалгаврыг гүйцэтгэх чадвартай. LLM-ийн гол цөм нь Transformer архитектур бөгөөд энэ нь self-attention механизмыг ашиглан текст дэх үгсийн хоорондын хамаарлыг алсын зайнаас үл хамааран тооцоолох боломжийг олгодог.

Google Gemini загвар нь мультимодал (multimodal) чадвартай буюу текст, зураг, аудио, видео зэрэг олон төрлийн өгөгдлийг нэгэн зэрэг боловсруулах чадвартай гэдгээрээ онцлог. Энэ төсөлд Gemini-1.5-Flash загварыг голчлон текст боловсруулах болон дүн шинжилгээ хийх (analytics report generation) зорилгоор ашигласан болно.

2.4.1 RAG (Retrieval-Augmented Generation) төстэй аргачлал

Систем нь цуглуулсан Facebook өгөгдлийн хураангуйг (summary) бүрдүүлж, түүнийгээ prompt болгон LLM-д өгдөг. Ингэснээр LLM нь ерөнхий мэдлэгээр биш, яг тухайн хуудасны сүүлийн үеийн бодит өгөгдөлд тулгуурлан дүгнэлт (insight) гаргадаг. Энэ нь "hallucination" буюу хиймэл оюуны хийсвэр төөрөгдлийг бууруулах ач холбогдолтой.

RAG архитектур нь ихэвчлэн вектор өгөгдлийн сан (Vector Database) болон embedding загваруудыг хослуулан ашигладаг. Гэвч энэхүү төслийн хувьд сүүлийн үеийн богино хэмжээний постуудыг анализд оруулж байгаа тул бүрэн хэмжээний RAG pipeline (FAISS, Pinecone зэрэг) ашиглахын оронд In-Context Learning буюу контекстэд шууд өгөгдлийг оруулах аргыг хэрэглэсэн. Энэ нь системийн архитектурыг энгийн байлгахын зэрэгцээ үр дүнг хурдан бөгөөд бодитой гаргах давуу талтай юм.

2.5 Бүлгийн дүгнэлт

Энэхүү бүлэгт нийгмийн сүлжээний өгөгдөл цуглуулах онолын үндэс, ижил төстэй шийдлүүдийн харьцуулалт болон NLP (Sentiment, Topic analysis), LLM (Gemini) ашиглан AI тайлан үүсгэх аргуудыг судалж тайлбарлав. Эдгээр онолын ойлголтууд нь дараагийн бүлгүүдэд системийн архитектур, шинжилгээ ба загварчлал хийх үндсэн суурь болох юм.

БҮЛЭГ 3

Системийн шинжилгээ ба
загварчлал

Энэ бүлэгт системийг хөгжүүлэхээс өмнө хийгдэх програм хангамжийн инженерийн шинжилгээ, хэрэглэгчийн шаардлага (Use Case), системийн ерөнхий архитектур, болон өгөгдлийн сангийн загварчлалыг дэлгэрэнгүй харуулна.

3.1 Хэрэглэгчийн шаардлага ба Use Case диаграм

Системийг ашиглах үндсэн хэрэглэгч нь Системийн админ (System Admin) болон Энгийн хэрэглэгч/Аналист (Analyst) байна. Тэдгээрийн системтэй харилцах үйлдлүүдийг Use Case диаграммаар харуулав.

3.1.1 Функциональ шаардлагууд

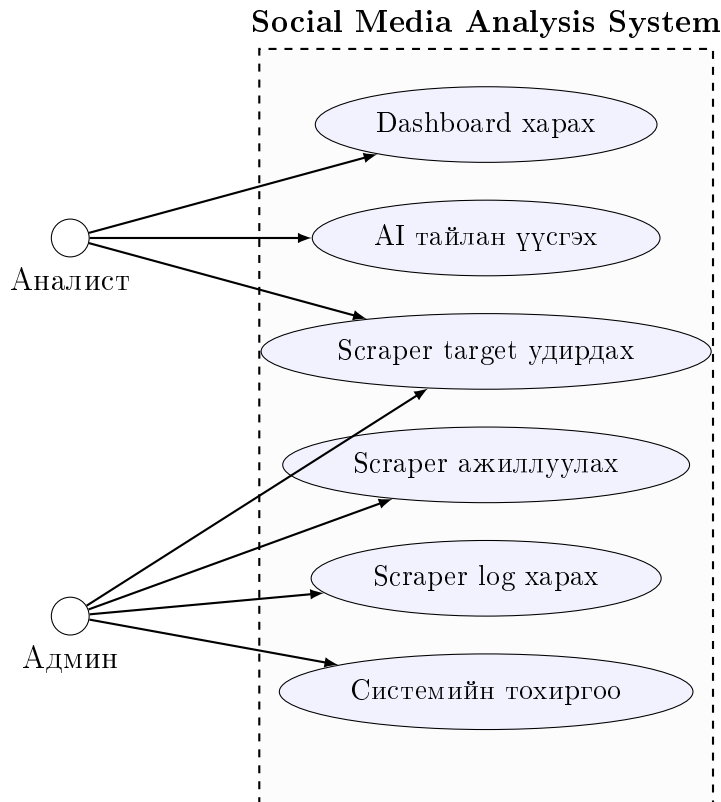
Системийн гүйцэтгэх үндсэн үүргүүдийг функциональ шаардлага (Functional Requirements) гэдэг. Энэхүү систем нь дараах шаардлагуудыг хангасан байх ёстой:

- **FR-01:** Систем нь хэрэглэгчийн өгсөн Facebook URL-аас пост болон сэтгэгдлүүдийг автоматаар цуглуулна.
- **FR-02:** Систем цуглуулсан өгөгдлийг өгөгдлийн санд давхардалгүй хадгална.
- **FR-03:** Систем цуглуулсан текстийн утгыг (sentiment) эерэг, сөрөг, саармаг гэж ангилна.
- **FR-04:** Систем нь өгөгдлөөс хамаарсан статистик дүгнэлтийг AI ашиглан гаргана.
- **FR-05:** Систем нь тайланг график болон хүснэгтээр Dashboard дээр харуулна.

3.1.2 Функциональ бус шаардлагууд

Функциональ бус шаардлагууд (Non-Functional Requirements) нь системийн гүйцэтгэл, найдвартай байдал, аюулгүй байдлыг тодорхойлдог:

- **NFR-01 (Гүйцэтгэл):** Dashboard-ийн хуудсууд болон API хүсэлтүүд нь 2 секундээс бага хугацаанд хариу өгдөг байх ёстой.
- **NFR-02 (Найдвартай байдал):** Scraper нь Facebook-ийн CAPTCHA болон rate limit-тэй тулгарсан үед процессыг автоматаар түр зогсоох эсвэл дахин оролдох (retry) логиктой байна.
- **NFR-03 (Аюулгүй байдал):** API endpoint-ууд болон AI тайлан үүсгэх хэсгүүд нь тохирох токен (Token) болон API Key-гээр баталгаажсан байна.
- **NFR-04 (Өргөтгөх боломж):** Системийн өгөгдлийн сангийн архитектур нь цаашид хэдэн сая бичлэг (records) хадгалахад гацахгүй байхаар индекслэгдсэн (indexed) байх шаардлагатай.



ЗУРАГ 3.1: Хэрэглэгчийн үүрэг ба Use Case диаграм

3.1.3 Use Case тодорхойлолтууд

Доорх хүснэгтүүдэд системийн гол үйлдлүүдийн (Use Cases) дэлгэрэнгүй тодорхойлолтыг харуулав.

Хүснэгт 3.1: UC-01: Dashboard харах

Үзүүлэлт	Тайлбар
Use case-ийн нэр	Dashboard харах
Оролцогч	Аналист, Админ
Зорилго	Цуглуулсан өгөгдлийн статистик, оролцооны хандлага (engagement trend)-г график хэлбэрээр харах
Урьдчилсан нөхцөл	Систем ажиллаж, backend API-тай холбогдсон байна.
Үндсэн урсгал	- Хэрэглэгч систем рүү нэвтэрч Dashboard цэсийг сонгоно. - Систем backend-ээс статистик мэдээллийг татаж авна. - Нийт пост, engagement, sentiment score зэрэг үзүүлэлтийг харуулна. - Хэрэглэгч refresh товч дарж мэдээллээ шинэчлэх боломжтой.
Альтернатив урсгал	3а. Хэрэв backend ажиллахгүй байвал систем "Demo fallback data" харуулж, backend унтарсан төлөвтэй байгааг мэдэгдэнэ.

Хүснэгт 3.2: UC-02: Scraper ажиллуулах

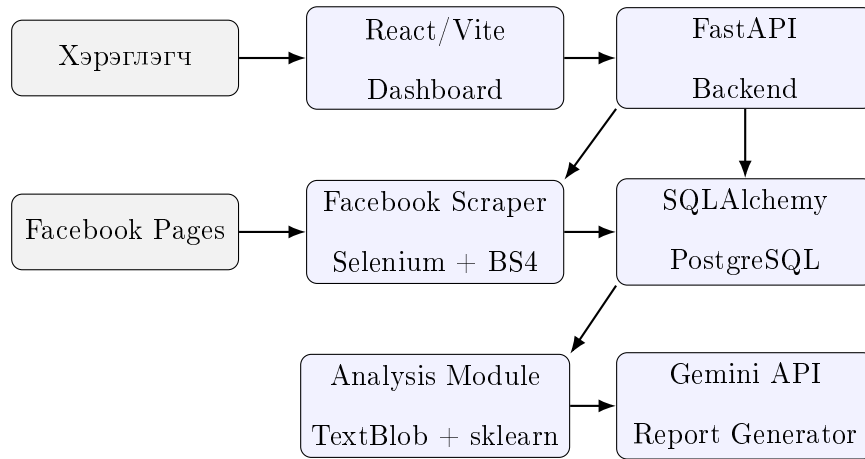
Үзүүлэлт	Тайлбар
Use case-ийн нэр	Scraper ажиллуулах
Оролцогч	Админ
Зорилго	Сонгосон Facebook хуудсуудаас дата цуглуулах процессыг эхлүүлэх
Урьдчилсан нөхцөл	pages.txt-д хамгийн багадаа нэг зорилтот хуудас (target) бүртгэгдсэн байна.
Үндсэн урсгал	- Админ Data Sources цэс рүү орно. "Start scraping" товчийг дарна. - Систем backend рүү хүсэлт илгээж Selenium scraper-ийг background-д ажиллуулна. - Scraper-ийн төлөв "Active" болж өөрчлөгдөнө.
Альтернатив урсгал	3а. Хэрэв өмнөх scraper process аль хэдийн ажиллаж байвал систем давхар эхлүүлэхээс татгалзаж анхааруулга өгнө.

Хүснэгт 3.3: UC-03: AI тайлан үүсгэх

Үзүүлэлт	Тайлбар
Use case-ийн нэр	AI тайлан үүсгэх
Оролцогч	Аналист, Админ
Зорилго	Сүүлийн үеийн постууд дээр үндэслэн хиймэл оюуны тусламжтай уншигдахуйц тайлан, дүгнэлт гаргуулах
Урьдчилсан нөхцөл	Gemini API түлхүүр тохируулагдсан байна.
Үндсэн урсгал	- Хэрэглэгч AI Analysis цэс рүү орж "Generate report" дарна. - Систем сүүлийн 50 постын хураангуйг бэлтгэж Gemini API рүү илгээнэ. - Gemini загвар нь өгөгдлийг анализ хийж Монгол хэлээр тайлан үүсгэнэ. - Систем тайланг дэлгэцэнд харуулна.
Альтернатив урсгал	2а. API түлхүүр буруу эсвэл хязгаар хэтэрсэн (quota exceeded) бол систем алдааны мессеж харуулна.

3.2 Системийн архитектурын загварчлал

Бодит систем нь frontend, backend, scraper, database, ML analysis, AI report гэсэн үндсэн давхаргатай. Төслийн гол урсгал нь Facebook target list удирдах, scraper ажиллуулах, PostgreSQL-д хадгалах, FastAPI-аар өгөгдөл буцаах, React dashboard дээр харуулах, шаардлагатай үед Gemini тайлан үүсгэх хэлбэртэй.



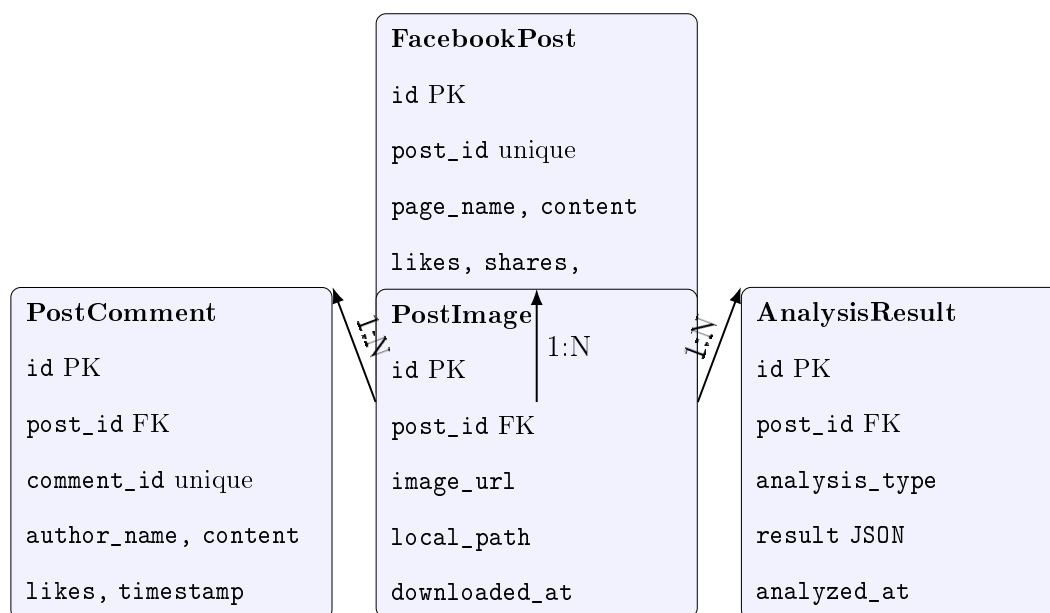
ЗУРАГ 3.2: Системийн логик архитектурын диаграм

Архитектур нь үүрэг зориулалтаараа дараах хэсгүүдэд хуваагдсан:

- **Presentation Layer (Frontend):** React 19, Vite, TypeScript, TailwindCSS ашиглан хэрэглэгчтэй харилцах интерфэйсийг бүрдүүлнэ.
- **Application Layer (Backend):** FastAPI ашиглан RESTful API үүсгэж, өгөгдөл дамжуулах, background process (scraper) удирдах логик ажиллана.
- **Data/Storage Layer:** PostgreSQL (үндсэн) болон SQLite (fallback) өгөгдлийн сангуудыг SQLAlchemy ORM ашиглан удирдана.
- **AI/Analytics Layer:** TextBlob, Scikit-learn (LDA) болон Google Gemini API-ууд хамтарч өгөгдөл дээр анализ хийнэ.

3.3 Өгөгдлийн сангийн загварчлал (ERD)

Системийн өгөгдлийн сан нь relational (харилцаат) бүтэцтэй. Үндсэн entity-үүд болох FacebookPost, PostComment, PostImage болон AnalysisResult хоорондын харилцааг доорх ERD диаграммаар үзүүлэв.



ЗУРАГ 3.3: Өгөгдлийн сангийн ERD диаграм

Харьцааны тайлбар:

- **1:N (Нэгээс-Олон):** Нэг пост олон сэтгэгдэлтэй (PostComment) байж болно.
- **1:N (Нэгээс-Олон):** Нэг пост олон зурагтай (PostImage) хавсрагдсан байж болно.
- **1:N (Нэгээс-Олон):** Нэг пост дээр Sentiment, Topic гэх мэт олон төрлийн шинжилгээний үр дүн (AnalysisResult) хийгдэж хадгалагдах боломжтой.

3.4 Процессын дарааллын загварчлал

3.4.1 Өгөгдөл цуглуулах процесс (Activity)

Scraper хэрхэн ажиллаж, өгөгдөл хэрхэн цуглуулагдаж хадгалагдах урсгалыг доорх алхмаар тодорхойлно:

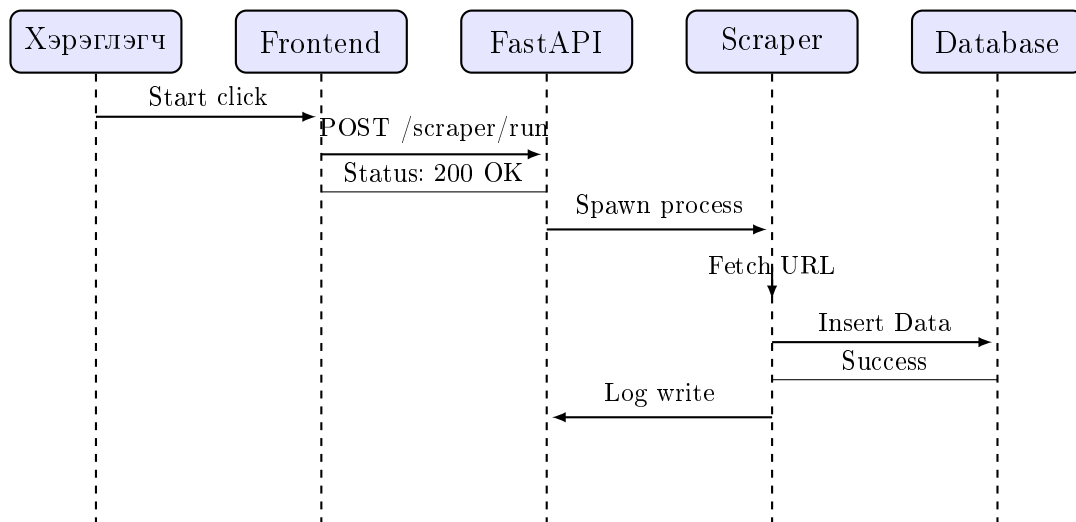
1. `pages.txt`-ээс Facebook page URL/ID жагсаалт унших.
2. Selenium browser session эхлүүлэх, шаардлагатай `directory/log` үүсгэх.

3. Target бүр рүү navigate хийж, scroll болон selector-based extraction хийх.
4. BeautifulSoup болон Selenium selector ашиглан post, image, comment data ялгах.
5. SQLAlchemy DatabaseManager.save_post() ашиглан duplicate (давхардал) шалгаж хадгалах.
6. Log файлыг /api/v1/scrapper/logs endpoint-оор frontend-д харуулах.

Энэхүү процесс нь тасралтгүй ажиллах боломжтой бөгөөд алдаа гарсан үед логт тэмдэглээд дараагийн target руу шилжих алдаа зохицуулах (error handling) механизмтай.

3.4.2 Дарааллын диаграм (Sequence Diagram)

Хэрэглэгч frontend-ээс "Scrapper ажиллуулах"команд өгөх үеийн дарааллыг (Sequence Diagram) дараах байдлаар тодорхойлов. Энэ нь объектууд хоорондоо хэрхэн, ямар дарааллаар мэдээлэл солилцож байгааг харуулдаг динамик загвар юм.



ЗУРАГ 3.4: Scraper ажиллуулах процессын дарааллын диаграм

3.4.3 Тайлан үүсгэх процесс (AI Generation)

Gemini API ашиглан тайлан гаргах процесс:

1. Frontend-ээс /gemini/analyze endpoint руу хүсэлт илгээнэ.
2. Backend нь хамгийн сүүлийн үеийн (latest) 50 постыг өгөгдлийн сангаас query хийж татна.
3. Татсан өгөгдлүүдээс чухал хэсгүүдийг (content, engagement score, sentiment) нэгтгэн JSON/Text prompt үүсгэнэ.
4. Prompt-ийг Gemini API руу илгээнэ.
5. Gemini-аас ирсэн Markdown хэлбэртэй тайланг Frontend руу буцааж, дэлгэцэнд render хийж харуулна.

3.5 Бүлгийн дүгнэлт

Энэхүү бүлэгт системийг хөгжүүлэхээс өмнө шаардагдах Use Case диаграм, үйл явцын урсгал, системийн ерөнхий болон өгөгдлийн сангийн архитектурыг тодорхойллоо. Загварчлал нь системийг модульчлагдсан (өгөгдөл цуглуулах, хадгалах, тайлагнах) байдлаар бие даан хөгжүүлэх боломжтойг харуулж байна.

БҮЛЭГ 4

Системийн хөгжүүлэлт

Энэхүү бүлэгт системийн шинжилгээ ба архитектурын загварчлалд үндэслэн хийгдсэн бодит програмчлалын хөгжүүлэлт, ашигласан технологийн шийдлүүдийг тайлбарлана.

4.1 Төслийн бүтэц

Төслийн root хэсэгт frontend, backend, Supabase migration, thesis material зэрэг хэсгүүд байрлана. Бодит application code нь дараах байдлаар зохион байгуулагдсан:

- `src/`: React/Vite frontend үндсэн код.
- `src/components/`: Dashboard, DataSources, PredictiveAnalysis, AdminControl болон UI компонентууд.
- `src/lib/`: Backend API client, Gemini client, Supabase client, mock data.
- `beta/`: FastAPI backend, scraper, database, ML, integrations агуулсан Python код.
- `beta/api_server.py`: REST API, health, stats, posts, analysis, scraper control endpoint-ууд.
- `beta/scraper/`: BrowserManager, PostScraper, EdgeFacebookScraper зэрэг scraper код.
- `beta/db/`: SQLAlchemy database manager, model, config.
- `beta/ml/`: DataAnalyzer, GeminiAnalyzer болон sentiment/topic analysis модулиуд.

4.2 Өгөгдлийн сангийн хөгжүүлэлт

Database хэрэгжилт нь SQLAlchemy ORM (Object-Relational Mapping) дээр суурилсан. `DatabaseManager` класс нь engine/session үүсгэх, table initialize хийх, post/comment/image/result хадгалах, query хийх, duplicate update хийх, health check хийх үүрэгтэй.

SQLAlchemy ашиглан хүснэгт үүсгэх логикийг дараах кодын хэсэгт харуулав:

```
from sqlalchemy import Column, String, Integer, DateTime
from sqlalchemy.orm import declarative_base

Base = declarative_base()

class FacebookPost(Base):
    __tablename__ = 'facebook_posts'

    id = Column(Integer, primary_key=True, autoincrement=True)
    post_id = Column(String, unique=True, index=True)
    page_name = Column(String)
    content = Column(String)
    likes = Column(Integer, default=0)
    comments = Column(Integer, default=0)
    shares = Column(Integer, default=0)
    timestamp = Column(DateTime)
```

Дээрх кодоод `FacebookPost` классыг тодорхойлж, өгөгдлийн сангийн хүснэгтийн багануудыг (columns) ORM ашиглан маппинг (mapping) хийсэн. Ингэснээр Python кодноос шууд SQL бичих шаардлагагүйгээр объектын түвшинд харилцах боломжтой болсон.

Гол шинжүүд:

- Existing `post_id` байгаа үед шинэчилж update хийх (Upsert logic).
- Comment duplicate-г `comment_id` эсвэл `author/content`-оор давхар шалгах.
- Image duplicate-г URL-аар шалгах.
- Analysis result-г `post_id + analysis_type` хосолсон түлхүүрээр update/save хийх.

4.3 Backend хөгжүүлэлт (FastAPI)

`beta/api_server.py` нь системийн гол REST API сервер юм. Backend startup үед database manager, analysis module, Google Sheets exporter, Gemini integration-ийг initialize хийхийг оролддог.

Хүснэгт 4.1: Backend endpoint-уудын үндсэн бүлэг

Endpoint бүлэг	Үүрэг
<code>/health</code>	Backend, database, analyzer, sheets, gemini initialization status шалгах
<code>/api/v1/posts, /posts</code>	Post list болон single post унших
<code>/posts/{id}/comments</code>	Post-ийн comments унших/үүсгэх
<code>/posts/{id}/images</code>	Post-ийн image records унших/үүсгэх
<code>/api/v1/stats</code>	Total posts, pages, comments, average engagement metrics буцаах
<code>/api/v1/sentiment</code>	Single/batch sentiment analysis ажиллуулах
<code>/api/v1/topics/trends</code>	Topic modeling болон keyword/hashtag trend extraction
<code>/api/v1/pages</code>	<code>pages.txt</code> файл унших/бичих (Scraping targets)
<code>/api/v1/scrapper/*</code>	Scrapper process start/status/logs удирдах
<code>/gemini/*</code>	Gemini summary, Q&A, topic detection

4.4 Scraper хөгжүүлэлт

Scraper хэсэг нь Selenium WebDriver болон BeautifulSoup ашиглан Facebook page content-ийг авахад чиглэсэн. `run_scraper.py` нь command-line entry point бөгөөд `-headless`, `-max-posts`, `-no-screenshots`, `-no-images`, `-debug`

сонголтуудтай. Backend scraper endpoint нь энэ script-ийг background process болгон эхлүүлдэг.

Selenium WebDriver ашиглан хөтөч эхлүүлэх тохиргооны жишээ:

```
from selenium import webdriver

from selenium.webdriver.firefox.options import Options

def setup_driver(headless=True):

    options = Options()

    if headless:

        options.add_argument("--headless")

        options.add_argument("--disable-gpu")

        options.add_argument("--window-size=1920,1080")

    driver = webdriver.Firefox(options=options)

    driver.implicitly_wait(10)

    return driver
```

Дээрх тохиргоо нь browser-ыг дэлгэцэнд харуулахгүйгээр цаана (headless mode) ажиллуулж, санах ойн ачааллыг бууруулах, сервер дээр шууд ажиллах нөхцөлийг бүрдүүлдэг.

Scraper workflow:

1. Log directory болон output directory үүсгэнэ.
2. `pages.txt`-ээс target list уншина.
3. Target бүр дээр AutoScraper ашиглан scrape хийнэ.
4. Амжилт/алдааг `logs/facebook_scraper.log` файлд бичнэ.
5. Frontend log endpoint-оор сүүлийн мөрүүдийг уншиж харуулна.

4.5 Frontend хөгжүүлэлт (React Dashboard)

Frontend нь React 19 болон Vite дээр бүтээгдсэн. `App.tsx` нь Dashboard, Data Sources, AI Analysis, Admin, Settings гэсэн үндсэн tab-уудыг удирдана.

Dashboard нь backend-ээс `/api/v1/posts` болон `/api/v1/stats` endpoint-уудаар өгөгдөл уншиж, дараах үзүүлэлтийг харуулдаг:

- Posts collected, engagement, estimated reach, average sentiment summary.
- Recharts ашигласан line/bar/area chart (Оролцооны хандлага).
- Latest signals буюу recent posts list.
- Backend live эсвэл demo fallback status badge.

4.6 AI тайлан болон ML шинжилгээний хөгжүүлэлт

4.6.1 Data Analyzer модуль

`DataAnalyzer` нь гурван үндсэн функцтэй:

- **Sentiment analysis:** `TextBlob` боломжтой үед polarity/subjectivity тооцоолно. Боломжгүй үед basic keyword matching fallback ажиллана.
- **Topic classification:** `CountVectorizer` болон LDA ашиглан top keywords бүхий topic label үүсгэнэ.
- **Engagement prediction:** Likes, shares, comments-оос оноо тооцоолж, data хангалттай үед `GradientBoostingRegressor` ашиглан score таамаглах оролдлого хийнэ.

4.6.2 Gemini AI тайлан

AI тайлан нь хоёр замаар хэрэгжсэн:

- **Frontend талд:** @google/genai ашиглаж gemini-2.5-flash model-д latest data summary prompt илгээдэг. Энэ нь хэрэглэгчийн browser-оос шууд AI-д хандах хөнгөн арга юм.
- **Backend талд:** google-generativeai ашигласан GeminiAnalyzer нь summary, insights, questions, improvement, critique төрлийн prompt-уудыг илгээдэг.

Хоёр тохиолдолд хоёуланд нь GEMINI_API_KEY орчны хувьсагчаар дамжин баталгаажилт (authentication) хийгдэнэ.

Frontend хэсэгт AI тайлан үүсгэх React component-ийн логикийг дараах байдлаар хэрэгжүүлсэн:

```
import { GoogleGenAI } from '@google/genai';

const ai = new GoogleGenAI({ apiKey: import.meta.env.VITE_GEMINI_API_KEY });

async function generateReport(promptText: string) {
  try {
    const response = await ai.models.generateContent({
      model: 'gemini-2.5-flash',
      contents: promptText,
    });
    return response.text;
  } catch (error) {
    console.error("Gemini API Error:", error);
    return null;
  }
}
```

Энэхүү код нь орчны хувьсагчаас (environment variables) API түлхүүрийг авч, gemini-2.5-flash загвар руу prompt илгээж, ирсэн текстийн хариуг

буцаана. Frontend нь уг буцаасан хариуг React Markdown компонент ашиглан дэлгэцэнд форматтайгаар харуулна.

4.7 Бүлгийн дүгнэлт

Энэхүү бүлэгт системийн backend (FastAPI), frontend (React), scraper болон өгөгдлийн сангийн бодит хөгжүүлэлтийн явц, техникийн шийдлүүдийг тайлбарлав. Дараагийн бүлэгт эдгээр хөгжүүлэлтийн үр дүнд бий болсон интерфэйс, туршилтын үр дүнгүүдийг дэлгэрэнгүй харуулна.

БҮЛЭГ 5

Хэрэгжилт ба үр дүн

5.1 Хэрэгжилтийн ерөнхий үр дүн

Төслийн бодит хэрэгжилтээр Facebook өгөгдөл цуглуулах, хадгалах, API-аар түгээх, dashboard дээр хянах, Gemini-ээр тайлан үүсгэх үндсэн workflow бүрэн бүрдсэн.

Хүснэгт 5.1: Хэрэгжсэн модулиудын нэгтгэл

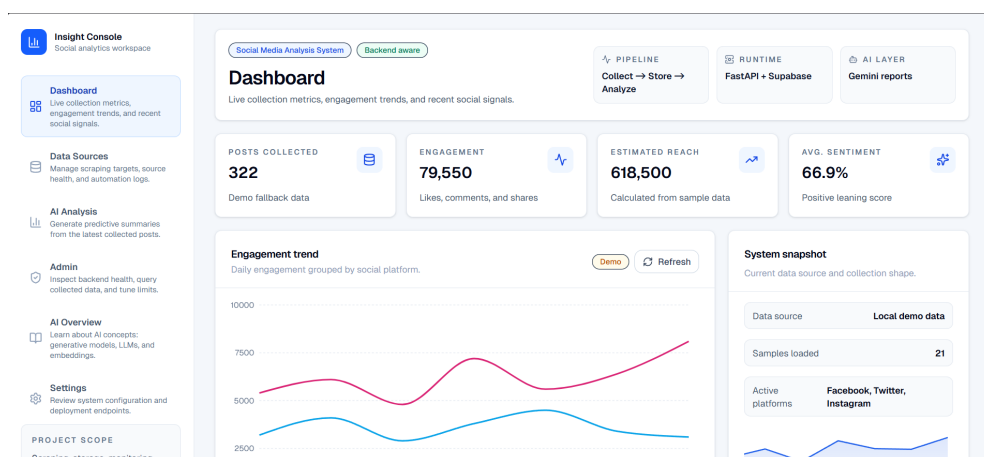
Модуль	Бодит үр дүн
Frontend dashboard	Live backend posts/statistics унших, demo fallback хяруулах, chart болон recent post cards гаргах
Data source manager	<code>pages.txt</code> target list унших/хадгалах, scraper run/status/log endpoint ашиглах
Admin control	Backend health, stats, posts snapshot хяруулах, browser-side keyword/date/platform filter хийх
FastAPI backend	Posts, comments, images, stats, sentiment, topics, trends, sheets export, Gemini, scraper control endpoint-ууд
Scraper	Facebook target list унших, Selenium/BeautifulSoup workflow ажиллуулах, log file үүсгэх
Database	PostgreSQL/SQLAlchemy model, duplicate-aware save, analysis result storage
Analysis	TextBlob/basic sentiment, LDA topic modeling, engagement score/prediction
AI report	Gemini API ашиглан recent data summary-аас Монгол хэл дээр тайлан, зөвлөмж үүсгэх

5.2 Хэрэглэгчийн интерфэйс болон функцүүдийн хэрэгжилт

Frontend нь хэрэглэгчид шууд харагдах үндсэн ажлын талбар болсон. Dashboard нь backend өгөгдөлтэй холбогдсон үед "Live mode", холбогдохгүй үед "Demo mode" гэж ялган харуулдаг. Энэ нь demo presentation болон backend availability хоёр нөхцөлийг нэг UI-аар даах боломж өгсөн.

5.2.1 Data Sources болон Scraper Control

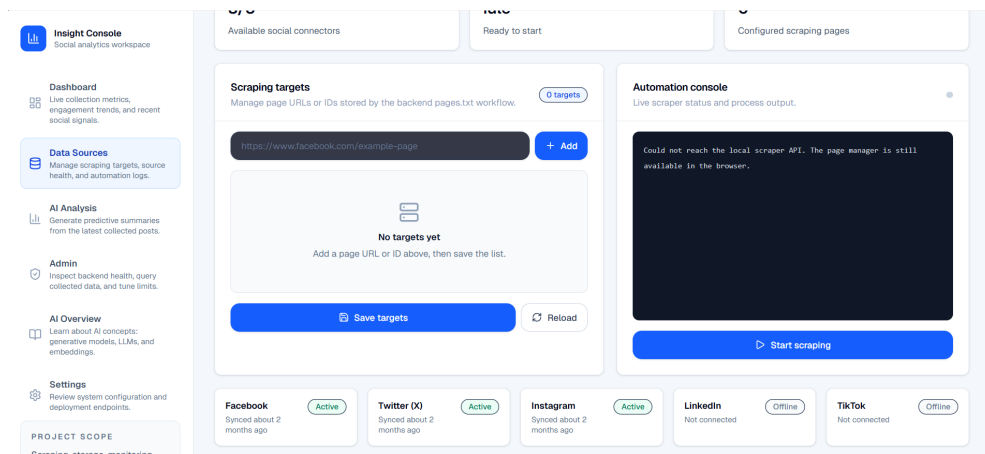
Хэрэглэгч Data Sources цонхонд нэвтэрч Facebook хуудасны URL оруулан scraping процессыг удирдана. Automation console нь scraper-ийн log-ийг шууд дэлгэцэнд харуулдаг.



ЗУРАГ 5.1: Data Sources цонх болон Automation Console

5.2.2 Үндсэн Dashboard болон Үзүүлэлтүүд

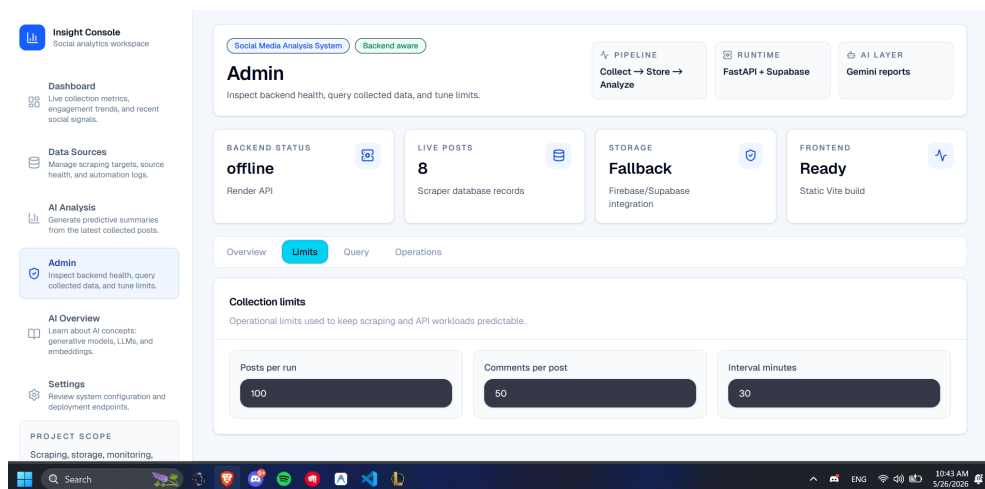
Dashboard хэсэгт нийт цуглуулсан постын тоо, engagement score, sentiment analysis-ийн дундаж үзүүлэлт зэргийг товчоор харуулна. Мөн хугацаанаас хамаарсан оролцооны графикийг (Engagement trend) Recharts ашиглан зурж харуулдаг.



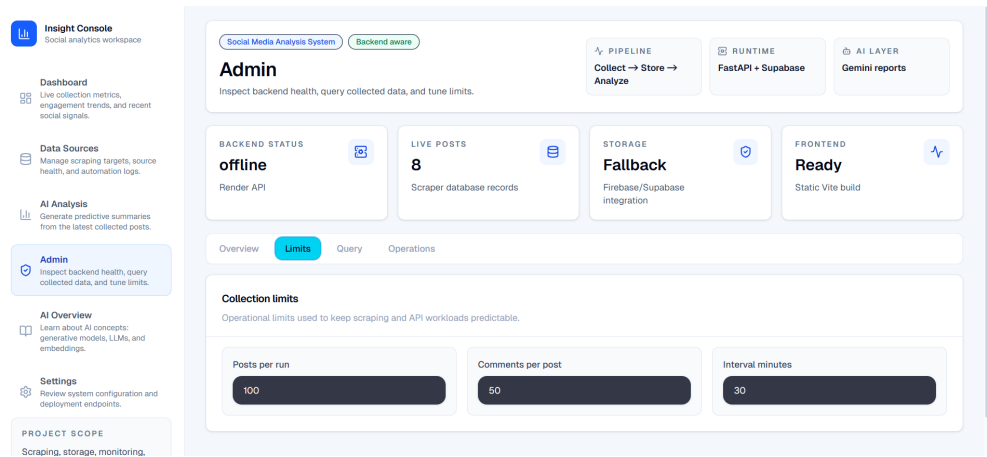
ЗУРАГ 5.2: Үндсэн Dashboard болон Engagement trend

5.2.3 Admin Control болон Хяналт

Admin panel нь системийн тохиргоо болон хязгаарлалтыг удирдах хэсэг юм. Эндээс backend server-ийн төлөв (status), өгөгдлийн сангийн холболт болон хэдэн пост, сэтгэгдэл татах limit зэргийг тохируулж болно.



ЗУРАГ 5.3: Admin Control болон Backend Status



ЗУРАГ 5.4: Collection limits (Хязгаарлалтын тохиргоо)

5.3 Туршилтын үр дүн ба баталгаажуулалт

Системийн функциональ шаардлагууд хэрхэн хангагдсаныг доорх байдлаар шалгаж баталгаажуулав.

Хүснэгт 5.2: API workflow баталгаажуулалт

Workflow	Үр дүн
Health check	Database/analyzer/sheets/gemini initialization flag амжилттай буцаана.
Posts/statistics	Dashboard-д хэрэгтэй post list, total posts, total pages, total comments, average metrics зөв буцаж байна.
Scraper control	Target list унших/бичих, scraper process эхлүүлэх, active status, log tail буцаах үйлдлүүд хэвийн ажиллаж байна.
Analysis	Sentiment, topic, engagement result-уудыг post бүр дээр тооцоолж мэдээллийн санд хадгалсан.
Gemini	Backend environment-д API key тохируулсан үед summary/Q&A prompt амжилттай ажиллаж тайлан гаргаж байна.

5.4 Үр дүнгийн хэлэлцүүлэг

Судалгааны эхэнд тавьсан асуултуудын хүрээнд: **RQ1-ийн хүрээнд:** Selenium болон BeautifulSoup ашигласан scraper workflow, target list, log management хэрэгжсэн тул Facebook өгөгдөл цуглуулах үндсэн pipeline амжилттай бүрдсэн. **RQ2-ийн хүрээнд:** FastAPI endpoint-ууд болон React dashboard нь live backend data-г chart, card, query view хэлбэрээр хэрэглэгчид хүргэж байна. Backend unavailable үед demo fallback ажилладаг нь presentation болон хөгжүүлэлтийн үед маш хэрэгтэй болсон. **RQ3-ийн хүрээнд:** TextBlob/basic sentiment, LDA topic modeling, engagement score, Gemini report generation нь өгөгдлийг зөвлөмж болгон хувиргах эхний шатны pipeline үүсгэсэн. Ингэснээр цуглуулсан хуурай өгөгдлийг тайлан болгож хувиргах автоматжуулалт бий боллоо.

5.5 Хязгаарлалт ба эрсдэл

- Facebook UI өөрчлөгдөхөд selector болон modal handling эвдрэх эрсдэлтэй.
- Backend болон frontend-д зарим текст encoding алдаа (Монгол фонто) гарч болзошгүй.
- Gemini report нь API key болон network availability-аас хамаарна.
- Бусад платформын (Instagram, TikTok) scraper болон deep-learning sentiment model нь энэ хувилбарт бүрэн хэрэгжээгүй.

БҮЛЭГ 6

Дүгнэлт

6.1 Үндсэн дүгнэлт

Энэхүү дипломын ажлын хүрээнд Facebook өгөгдлийг цуглуулах, хадгалах, API-аар дамжуулах, dashboard дээр хянах, Gemini ашиглан тайлан үүсгэх системийг амжилттай боловсруулсан. Судалгаа болон хөгжүүлэлтийн үр дүнд дараах зорилтууд хангагдлаа:

1. **Facebook scraper workflow:** `pages.txt`-д хадгалсан target list-ээр scraper ажиллуулах, log үүсгэх, backend-ээс scraper start/status/log хянах боломж бүрдсэн.
2. **REST API:** FastAPI backend нь health, posts, comments, images, stats, sentiment, topics, trends, export, Gemini, scraper control endpoint-уудтай болсон.
3. **Database layer:** PostgreSQL/SQLAlchemy model болон duplicate-aware save logic хэрэгжиж өгөгдлийг алдаагүй, давхардалгүй хадгалах боломжтой болсон.
4. **Frontend dashboard:** React/Vite dashboard нь live backend data, fallback demo data, chart, recent posts, admin query, scraper controls, Gemini report UI-г нэгтгэсэн цогц хяналтын самбар болсон.
5. **Analysis pipeline:** TextBlob/basic sentiment, LDA topic modeling, engagement scoring, Gemini report generation ажиллах бүтэцтэй болсон.

6.2 Шинэлэг тал

- Scraper target management, process control, log viewer-ийг command line-аас гаргаж, frontend UI-тай шууд холбосон.
- Backend live data болон demo fallback-ийг нэг dashboard-д нэгтгэсэн бөгөөд API унасан үед ч систем ажиллах чадвартай (Resilience).
- Relational storage, API, dashboard, AI report generation-ийг дипломын нэг системийн workflow болгон хэрэгжүүлсэн.

- Analysis result-г database-д JSON хэлбэрээр хадгалах боломжтой болгож, post-level analysis-г дараа нь дахин ашиглах нөхцөл бүрдүүлсэн.

6.3 Цаашдын хөгжүүлэлтийн чиглэл

Хэдийгээр систем үндсэн зорилгоо биелүүлсэн ч илүү сайжруулах, өргөжүүлэх боломжууд дараах байдалтай байна:

1. Монгол хэлний labeled dataset бүрдүүлж sentiment model тусгайлан сургаж нэвтрүүлэх.
2. Admin limit/toggle controls-ийг backend configuration болон database-д persist хийх (Одоогоор UI төлөв байдалтай байгаа).
3. Scraper selector test болон regression test нэмэх, Instagram/TikTok зэрэг өөр эх сурвалжуудыг дэмждэг болгох.
4. Labeled benchmark dataset ашиглан sentiment/topic/engagement model-ийн бодит accuracy болон F1 score хэмжиж загварыг сайжруулах.
5. Scraper ажиллах үед queue/job status table нэмж олон хэрэглэгчийн үед process control-ийг (жишээ нь Celery эсвэл Redis ашиглан) сайжруулах.

Энэхүү ажил нь өгөгдөл цуглуулах, хадгалах, харах, шинжлэх, AI тайлан болгох алхмуудыг нэг usable системд холбосонд үндсэн үнэ цэн оршино. Цаашид дээрх сайжруулалтуудыг хийснээр системийг судалгааны болон байгууллагын хэрэглээнд илүү тогтвортой ашиглах бүрэн боломжтой.

ХАВСРАЛТ А

Хавсралт

А.1 Системийн тохиргооны жишээ

Туршилтын орчинд дараах үндсэн тохиргоог хэрэглэсэн: Python орчин, Firefox WebDriver, өгөгдлийн сангийн холболт, API токен, скрапингийн delay параметрууд. Эдгээр тохиргоо нь туршилтыг дахин гүйцэтгэхэд шаардлагатай суурь нөхцөлийг бүрдүүлнэ.

А.2 API Endpoint жишээ

Системд ашигласан гол endpoint-уудыг дараах байдлаар бүлэглэж болно.

Хүснэгт А.1: Хавсралтын endpoint жагсаалт

Endpoint	Зориулалт
/health	Backend болон dependency status шалгах
/api/v1/posts, /api/v1/stats	Dashboard-ийн үндсэн өгөгдөл унших
/api/v1/pages	Scraper target list унших, хадгалах
/api/v1/scraper/run, /api/v1/scraper/ status, /api/v1/scraper/logs	Scraper ажиллуулах, төлөв болон log харах
/api/v1/sentiment, /api/v1/topics, /api/v1/trends	Analysis workflow ажиллуулах
/gemini/analyze, /gemini/ask	AI тайлан болон асуулт-хариулт үүсгэх
/export/sheets	Google Sheets export хийх

API Response JSON жишээ (FastAPI):

```
{
  "status": "success",
  "data": {
    "total_posts": 150,
    "total_comments": 432,
    "average_sentiment": 0.45,
    "recent_posts": [
      {
        "post_id": "123456789",
        "page_name": "TechNews Mongolia",
        "content": "Шинэ AI технологийн талаарх дэлгэрэнгүй мэдээлэл...",
        "likes": 120,
        "comments": 45
      }
    ]
  }
}
```

А.3 Өгөгдлийн сангийн схем

Өгөгдлийн сангийн үндсэн схем нь `FacebookPost`, `PostComment`, `PostImage`, `AnalysisResult` хүснэгтүүд дээр төвлөрсөн. `FacebookPost` нь үндсэн контент болон engagement metric, `PostComment` нь сэтгэгдлийн өгөгдөл, `PostImage` нь дагалдах медиа мэдээлэл, `AnalysisResult` нь sentiment/topic/engagement шинжилгээний JSON үр дүнг хадгална.

ХАВСРАЛТ В

Эх кодын жагсаалт

Энэхүү хавсралтад системийн хөгжүүлэлтэд ашигласан голлох эх кодуудыг хавсаргав.

В.1 Үндсэн API сервер (api_server.py)

Системийн backend хэсэг буюу FastAPI дээр бичигдсэн үндсэн сервер.

```
from fastapi import FastAPI, BackgroundTasks, HTTPException, Depends
from pydantic import BaseModel
import uvicorn
import os
import json
import subprocess
from datetime import datetime
from sqlalchemy.orm import Session
from sqlalchemy import func

from db.database import DatabaseManager, FacebookPost
from ml.analyzer import DataAnalyzer
from ml.gemini_integration import GeminiAnalyzer
from ml.sheets_integration import SheetsExporter

app = FastAPI(title="Social Media Analysis API", version="1.0.0")
db_manager = DatabaseManager()
analyzer = DataAnalyzer()
gemini = GeminiAnalyzer()

@app.get("/health")
def health_check():
    return {
```

```
        "status": "ok",

        "database": db_manager.engine is not None,

        "analyzer": analyzer.is_ready,

        "gemini": gemini.is_ready

    }

@app.get("/api/v1/stats")
def get_stats():
    session = db_manager.get_session()

    try:

        total_posts = session.query(FacebookPost).count()

        total_comments = session.query(func.sum(FacebookPost.comments)).scalar()

        avg_engagement = session.query(func.avg(
            FacebookPost.likes + FacebookPost.comments + FacebookPost.shares
        )).scalar() or 0

    return {

        "status": "success",

        "data": {

            "total_posts": total_posts,

            "total_comments": int(total_comments),

            "avg_engagement": round(float(avg_engagement), 2)

        }

    }

except Exception as e:

    raise HTTPException(status_code=500, detail=str(e))

finally:

    session.close()
```

```
@app.post("/api/v1/scrapper/run")
def run_scraper(background_tasks: BackgroundTasks):
    try:
        subprocess.Popen(["python", "run_scraper.py", "--headless", "--max-post
        return {"status": "success", "message": "Scrapper started in background
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

B.2 React Dashboard (App.tsx)

Системийн frontend интерфэйсийг удирддаг үндсэн React компонент.

```
import React, { useState, useEffect } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Sidebar from './components/Sidebar';
import Dashboard from './components/Dashboard';
import DataSources from './components/DataSources';
import AIAnalysis from './components/AIAnalysis';
import Settings from './components/Settings';

function App() {
    const [stats, setStats] = useState(null);

    useEffect(() => {
        fetch('/api/v1/stats')
            .then(res => res.json())
            .then(data => setStats(data.data))
            .catch(err => console.error("Failed to load stats", err));
    }, []);
```

```
    return (

        <BrowserRouter>

            <div className="flex h-screen bg-gray-50">

                <Sidebar />

                <main className="flex-1 overflow-y-auto p-8">

                    <Routes>

                        <Route path="/" element={<Dashboard stats={stats} />} />

                        <Route path="/sources" element={<DataSources />} />

                        <Route path="/analysis" element={<AIAalysis />} />

                        <Route path="/settings" element={<Settings />} />

                    </Routes>

                </main>

            </div>

        </BrowserRouter>

    );
}

export default App;
```

B.3 ML Analyzer (analyzer.py)

Өгөгдөлд NLP шинжилгээ хийдэг гол класс.

```
import pandas as pd

from textblob import TextBlob

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.decomposition import LatentDirichletAllocation

import re

class DataAnalyzer:

    def __init__(self):
```



```
self.is_ready = True

def analyze_sentiment(self, text):
    if not text:
        return 0.0
    blob = TextBlob(str(text))
    return blob.sentiment.polarity

def extract_topics(self, documents, n_topics=3):
    if len(documents) < 5:
        return ["Insufficient data for topic modeling"]

    vectorizer = CountVectorizer(stop_words='english', max_features=1000)
    doc_term_matrix = vectorizer.fit_transform(documents)

    lda = LatentDirichletAllocation(n_components=n_topics, random_state=42)
    lda.fit(doc_term_matrix)

    feature_names = vectorizer.get_feature_names_out()
    topics = []
    for topic_idx, topic in enumerate(lda.components_):
        top_words = [feature_names[i] for i in topic.argsort()[::-6:-1]]
        topics.append(f"Topic {topic_idx+1}: {' '.join(top_words)}")

    return topics
```